Vehicle Parking App – V1

Author :
Name: Anjali Rameshwar Kokare
Roll Number: 24f2002015
Student Email: 24f2002015@ds.study.iitm.ac.in

About Me

I am a second-year Data Science student at IIT Madras, passionate about web development and data-driven solutions. This project gave me hands-on experience in building a full-stack web application to solve a real-world parking management problem for both urban and rural areas. It helped me understand how to manage backend logic, handle dynamic database queries, and create interactive user interfaces with modern web technologies.

Project Description

The goal of this project was to build a Flask-based Vehicle Parking Management System that enables users to register, book, and release parking spots in real-time. Admins can manage parking lots, control spot allocation, and view comprehensive analytics through charts. The system is designed to be scalable, user-friendly, and secure, with distinct dashboards for users and admins.
The application includes features like user authentication, spot reservation, history tracking, and real-time spot status. Admins can dynamically create parking lots, manage city-wise spot distribution, and visualize booking statistics.

Technologies Used

Technology	Purpose
Python	Core backend logic, routing, and server-side operations
Flask	Web framework used to structure routes and handle HTTP requests
Flask-SQLAlchemy	ORM used to interact with SQLite, define models and relationships

SQLite	Lightweight, file-based database for local development
HTML5	Base structure of web pages
CSS3 & Bootstrap 5	Styling and responsive layouts
Jinja2	Template engine to dynamically render HTML with backend data
Matplotlib	To generate charts and visualize parking data (usage, revenue, etc.)

Database Schema Design

Relationships
One-to-Many:
One Parking_Lot → many Parking_Spots
One User → many Reservations or
Bookings Many-to-One:
Reservation belongs to one User, one Lot, and one Spot

Database Tables

User : id, name, email, password, address, pin_code

Admin : id, email, password

Parking_Lot : id, prime_location_name, price, address, pin_code, city, maximum_number_of_spots

Parking_Spot : id, lot_id (FK), spot_name, status, place, city

Reservation : id, spot_id (FK), user_id (FK), lot_id (FK), vehicle_number, parking_timestamp, leaving_timestamp, parking_cost,city ,place

Booking: id, vehicle_number, city, place, spot_name, lot_id, user_id, vehicle_type

Development Process

Planning:

Designed the database schema and split user and admin flows. Defined what data needs to be stored for booking, parking lots, and spot tracking.

Backend Setup:

Implemented route handling using Flask and database operations using SQLAlchemy. Defined all models and ensured proper foreign key relations.

Frontend Design:

Created templates using HTML, Bootstrap, and Jinja2. User dashboard, booking form, and admin panel were styled for clarity and responsiveness.

Interactive Features:

Booking auto-assigns the first available spot.
Admin dashboards use charts for analytics using Matplotlib.
Integrated spot status logic: green (available) / red (occupied).

Testing & Debugging:

Validated CRUD operations, tested spot release and reallocation, verified admin editing capabilities. Used sessions and flash messages for feedback.

Key Features

User Dashboard:

- View all available parking spots in chosen city/place
- Book a parking spot (auto-assigned)
- Release/cancel a booking
- View past reservations and costs
- See a graph of total parking usage/cost

Admin dashboard:

- Add/edit/delete parking lots
- Adjust number of spots dynamically
- Monitor real-time spot status (color-coded)
- View list of users, reservations, and revenue
- Graphical insights using Matplotlib charts

Charts and Analytics

Charts are generated using Matplotlib and embedded in admin/user dashboards.
Some examples:

- Revenue by parking lot
- Number of bookings per day
- Spot occupancy percentage
- User-wise usage history

Challenges Faced:

Model Relationships: Designing relationships with SQLAlchemy required a good understanding of foreign key logic.

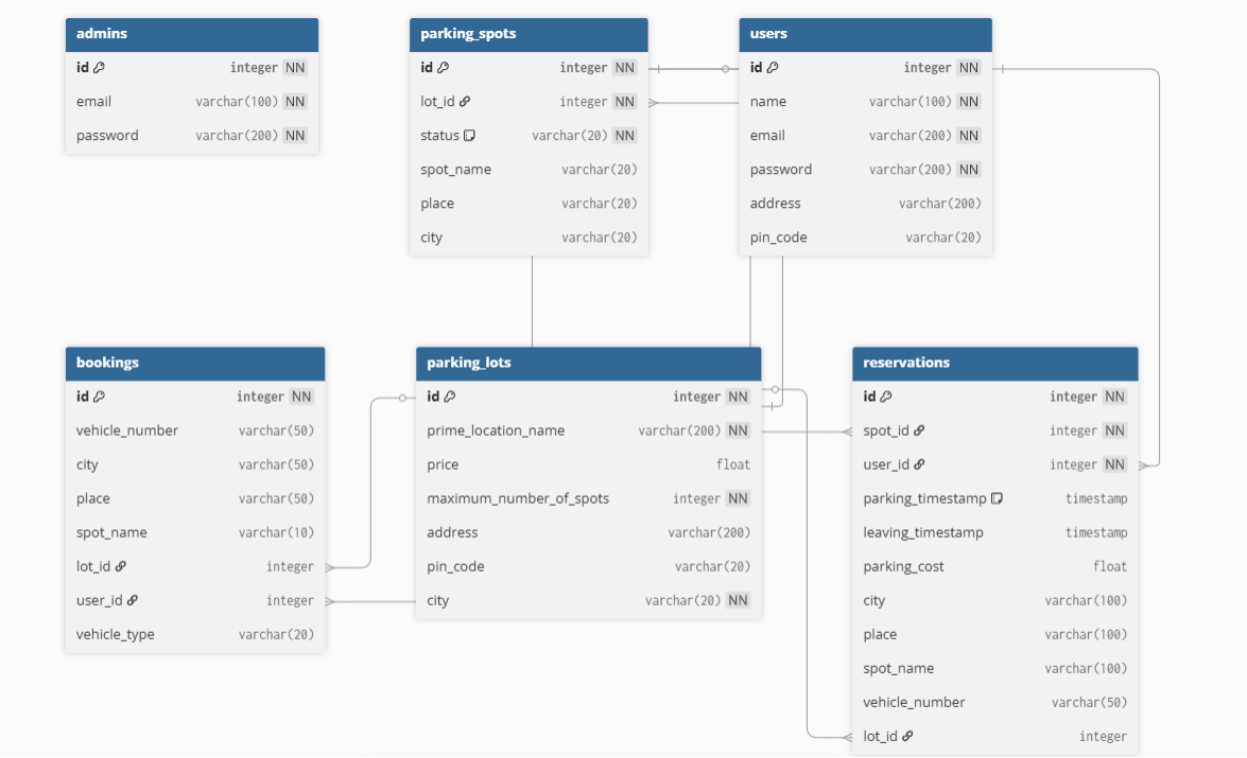
Spot Allocation Logic: Assigning the first available spot dynamically while handling concurrency was tricky.

Dynamic Admin Editing: Admins can change the number of spots, requiring careful database updates.

Graph Integration: Passing data from Flask to Matplotlib and rendering it inside templates took effort.

User Flow Control: Ensuring booking flow doesn't break with refresh or page reload needed session management.

Design Rationale



Note

"AI tools like ChatGPT were used for understanding tricky errors, optimizing certain complex parts of the code, and improving my conceptual clarity in Flask and frontend structure.

.

```
vehicle_parking_app/
├── app.py
├── backend/
│   ├── controller.py
│   └── models.py
├── instance/
│   └── parking_data.db
├── static/
│   ├── css/
│   └── images/
├── templates/
└── backend.zip
```

Demo Video

[Video Link] : **video for vehicle parking app**
